

Machine Learning

Welcome to the course

When: Monday and Thursday, 17:30-21:30

Format: Theoretical Material, Practical Material & Class Assignment

Requirements: Active participation, Class exercises, Final project

Topics:

- Exploratory Data Analysis (EDA)
- Wide range of predictive models
- Applied Machine Learning

Who am I?



B.Sc. in Information Systems Engineering - Technion

M.Sc. in Data Science - Technion

Summer Internship at Cornell Tech

Fourth year of teaching, second cohort of this course

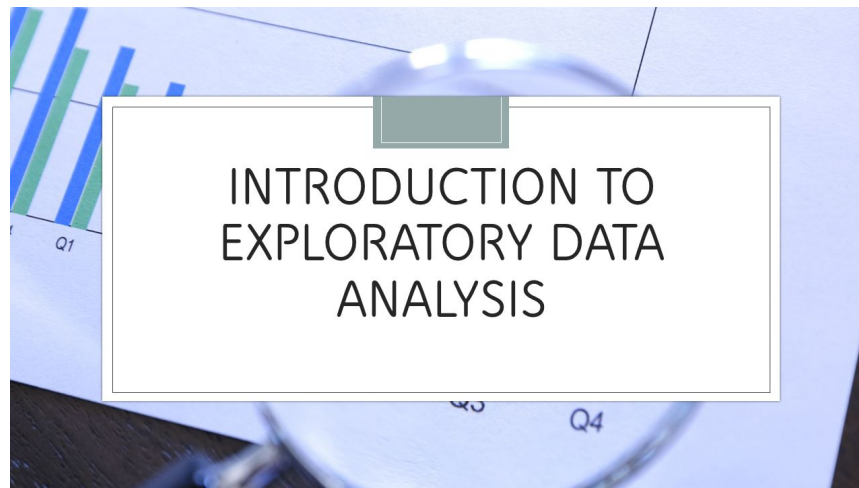
NLP Researcher - Focus on Explainability

Mail: Gilatt@campus.technion.ac.il

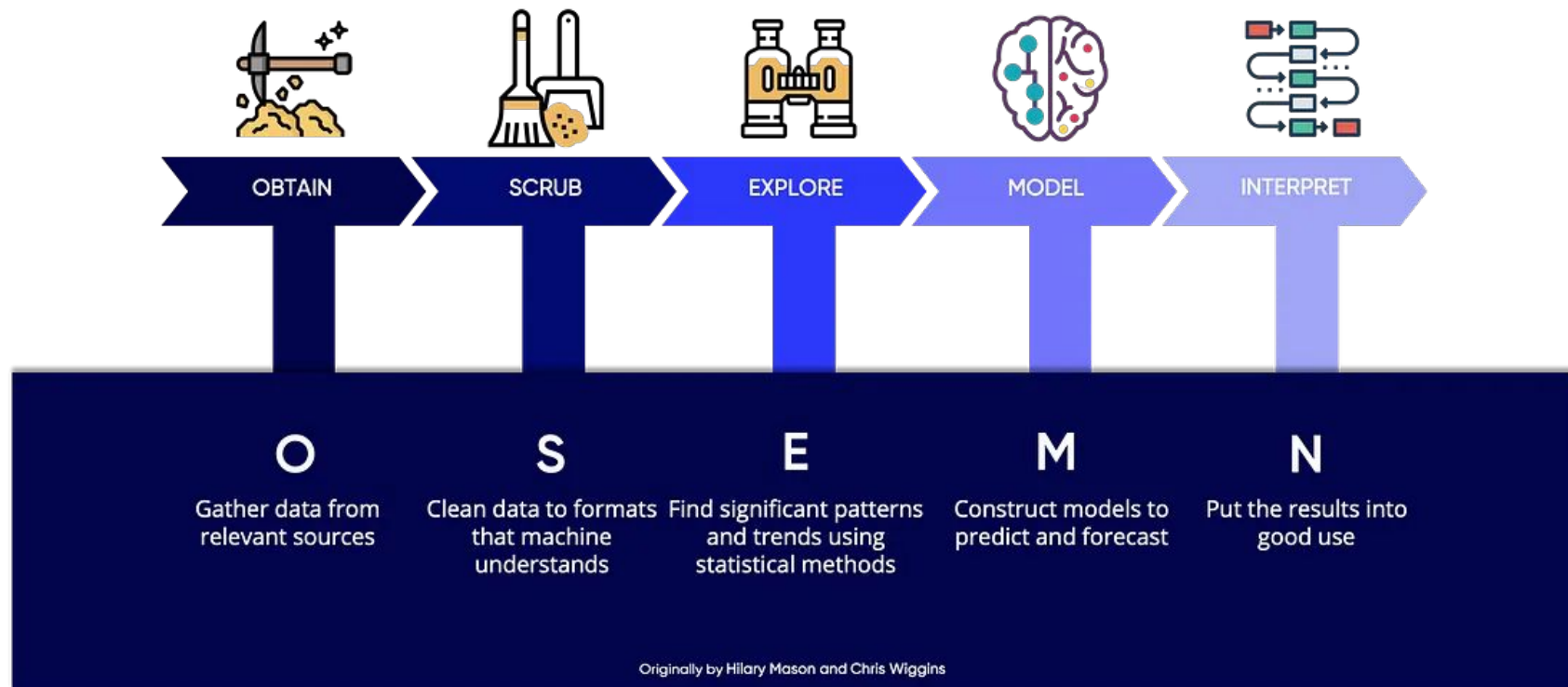
Machine learning

Lecture 1 - Exploratory Data Analysis (EDA)

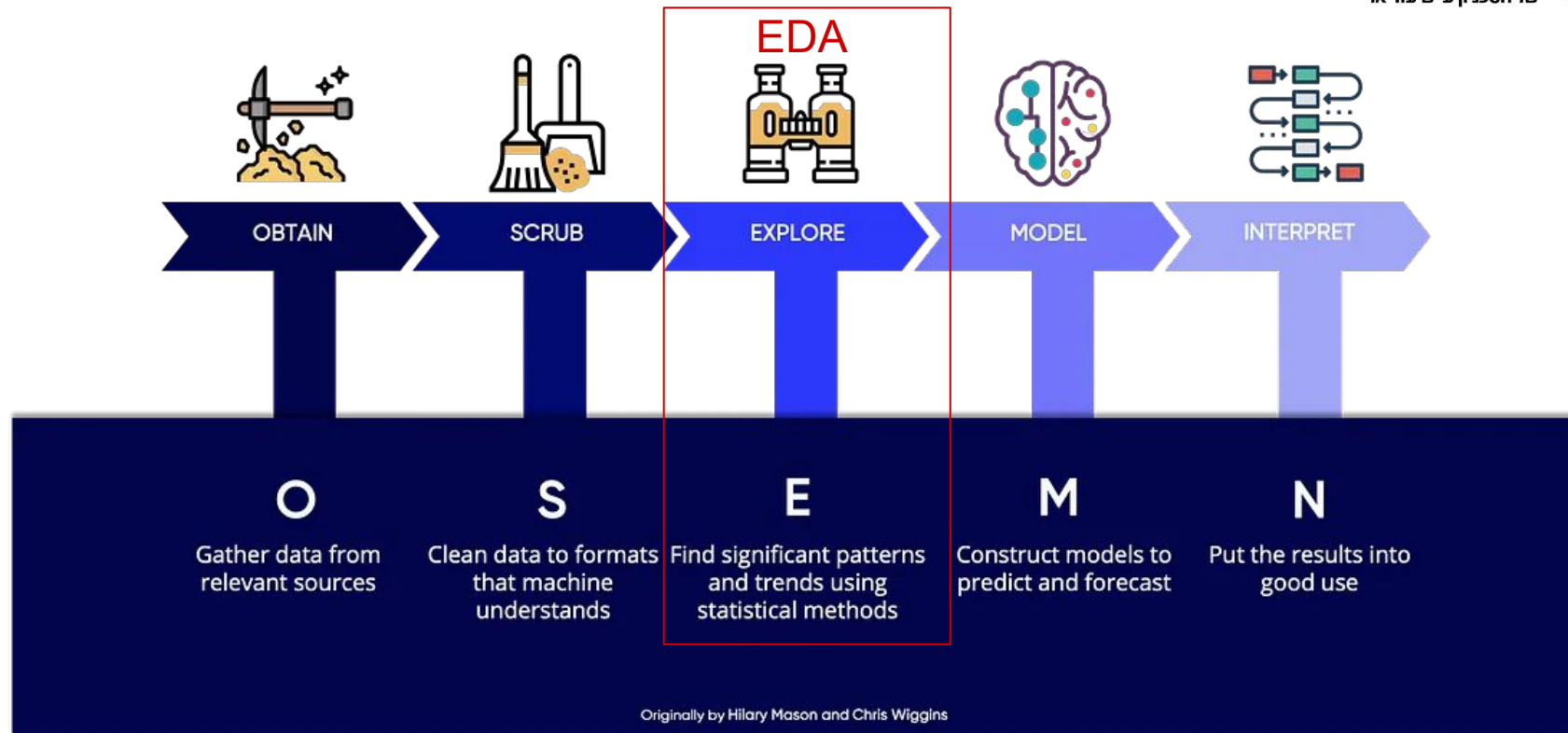
31.03.2025
Toker Gilat



Data Science Process



Data Science Process



Typical EDA

1. **Inspect data types:** Identify data types (e.g., numerical, categorical) and apply preprocessing techniques (e.g., scaling numerical features, encoding categorical variables).
2. **Calculate descriptive statistics:** Find measures of central tendency (e.g., mean, median) and variability (e.g., sd). Identify correlations to explore relationships between variables.
3. **Evaluate the distribution shape:** Analyze variable distributions to see if they follow expected patterns (e.g., normal distribution) are skewed (e.g., use log transformation).
4. **Utilize data visualization:** Graphs like histograms, box plots, and scatter plots reveal patterns, outliers, and relationships.
5. **Detect inconsistencies and anomalies:** Identify missing data, outliers, and mixed data types to decide if they need correction or removal.
6. **Assess the representation of each class:** Check if there are enough examples for each class (e.g., handle imbalanced datasets with oversampling).
7. **Feature engineering:** Create or transform features to improve data representation (e.g., interaction terms or aggregating temporal data).

Typical EDA

1. **Inspect data types:** Identify data types (e.g., numerical, categorical) and apply preprocessing techniques (e.g., scaling numerical features, encoding categorical variables).

EDA - demonstration

Inspect data types

	Categorical (ordinal)	Categorical (nominal)	Discrete	Continuous
Type	Alphanumeric	Alphanumeric	Numeric	Numeric
Number of values	Finite	Finite	Finite	Infinite
Hierarchical	Yes	No	-	-
Example	• Satisfaction ratings • Mood settings • Pain severity	• Gender • Payment method • Fruit type	• Number of customer complaints • Number of flaws or defect	• Distance • Weight

EDA - demonstration

Inspect data types

	Categorical (ordinal)	Categorical (nominal)	Discrete	Continuous
Type	Alphanumeric	Alphanumeric	Numeric	Numeric
Number of values	Finite	Finite	Finite	Infinite
Hierarchical	Yes	No	-	-
Example	• Satisfaction ratings • Mood settings • Pain severity	• Gender • Payment method • Fruit type	• Number of customer complaints • Number of flaws or defect	• Distance • Weight

EDA - demonstration

Inspect data types

Encoding Categorical Features

When working with categorical data in machine learning, it's essential to **convert categorical features into numerical values** that the model can understand.

There are different methods to achieve this, depending on whether the categorical data is **nominal** or **ordinal**.

EDA - demonstration

Inspect data types

Nominal data

Definition: Nominal data consists of categories without any intrinsic order.

Example: Categories like colors—"Red," "Blue," and "Green"—have no ranking or order.

Encoding Method: For nominal data, **One-Hot Encoding** is used. This method creates binary columns for each category to avoid implying any unintended order.

EDA - demonstration

Inspect data types

```
1 import pandas as pd
2
3 data = {'color': ['Red', 'Blue', 'Green', 'Blue', 'Red', 'Green']}
4 df = pd.DataFrame(data)
5 print("Original DataFrame:")
6 df
```

Original DataFrame:

	color	
0	Red	
1	Blue	
2	Green	
3	Blue	
4	Red	
5	Green	

EDA - demonstration

Inspect data types

```
from sklearn.preprocessing import OneHotEncoder
phe = OneHotEncoder()
encoded_features = phe.fit_transform(df[['color']]).toarray()
df_encoded = pd.DataFrame(encoded_features, columns=phe.get_feature_names_out(['color']))
df_ohe = pd.concat([df, df_encoded], axis=1)
print("\nOne-Hot Encoded DataFrame:")
df_ohe
```

One-Hot Encoded DataFrame:

	color	color_Blue	color_Green	color_Red
0	Red	0.0	0.0	1.0
1	Blue	1.0	0.0	0.0
2	Green	0.0	1.0	0.0
3	Blue	1.0	0.0	0.0
4	Red	0.0	0.0	1.0
5	Green	0.0	1.0	0.0

EDA - demonstration

Inspect data types

```
# Using pandas get_dummies
df_dummies = pd.get_dummies(df, columns=['color'],dtype=int)
print("\nDataFrame with get_dummies:")
df_dummies
```

DataFrame with get_dummies:

	color_Blue	color_Green	color_Red
0	0	0	1
1	1	0	0
2	0	1	0
3	1	0	0
4	0	0	1
5	0	1	0

EDA - demonstration

Inspect data types

Ordinal Data

Definition: Ordinal data consists of categories with a meaningful order or ranking.

Example: Categories like satisfaction —“Low,” “Medium,” and “High”.

Encoding Method: For ordinal data, **Label Encoding** is appropriate as it assigns numerical values in a way that reflects the order of categories.

EDA - demonstration

Inspect data types

```
import pandas as pd

data = {'quality': ['Low', 'High', 'Medium', 'Medium', 'High', 'Low']}
df_ordinal = pd.DataFrame(data)
print("Original DataFrame:")
df_ordinal
```

Original DataFrame:

	quality	
0	Low	
1	High	
2	Medium	
3	Medium	
4	High	
5	Low	

EDA - demonstration

Inspect data types

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df_ordinal['quality_encoded'] = le.fit_transform(df_ordinal['quality'])
print("\nLabel Encoded DataFrame with Ordinal Categories:")
df_ordinal
```

Label Encoded DataFrame with Ordinal Categories:

	quality	quality_encoded	
0	Low	1	
1	High	0	
2	Medium	2	
3	Medium	2	
4	High	0	
5	Low	1	

EDA - demonstration

Inspect data types

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df_ordinal['quality_encoded'] = le.fit_transform(df_ordinal['quality'])
print("\nLabel Encoded DataFrame with Ordinal Categories:")
df_ordinal
```

Label Encoded DataFrame with Ordinal Categories:

	quality	quality_encoded
0	Low	1
1	High	0
2	Medium	2
3	Medium	2
4	High	0
5	Low	1

The assignment of numerical values to categories is based on the lexicographical order of the category names. The LabelEncoder from scikit-learn sorts the unique category names and assigns integers starting from 0.

EDA - demonstration

Inspect data types

Mapping:

```
ordinal_mapping = {'Low': 0, 'Medium': 1, 'High': 2}

# Apply the custom mapping
df_ordinal['quality_encoded'] = df_ordinal['quality'].map(ordinal_mapping)
print("\nCustom Ordinal Encoded DataFrame:")
df_ordinal
```

Custom Ordinal Encoded DataFrame:

	quality	quality_encoded	
0	Low	0	
1	High	2	
2	Medium	1	
3	Medium	1	
4	High	2	
5	Low	0	

Typical EDA

1. **Inspect data types:** Identify data types (e.g., numerical, categorical) and apply preprocessing techniques (e.g., scaling numerical features, encoding categorical variables).
2. **Calculate descriptive statistics:** Find measures of central tendency (e.g., mean, median) and variability (e.g., sd). Identify correlations to explore relationships between variables.

EDA - demonstration

Calculate descriptive statistics

```
fruits.select_dtypes(include=['object']).describe()
```

	fruit_name	fruit_subtype
count	59	59
unique	4	10
top	apple	unknown
freq	19	10

Most common value

Frequency of the top value

* For numeric data, the result includes: count, mean, std, min, max, lower & upper percentiles

EDA - demonstration

Calculate descriptive statistics

```
fruits.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 59 entries, 0 to 58
Data columns (total 7 columns):
fruit_label      59 non-null int64
fruit_name       59 non-null object
fruit_subtype    59 non-null object
mass             59 non-null int64
width           59 non-null float64
height          59 non-null float64
color_score      59 non-null float64
dtypes: float64(3), int64(2), object(2)
memory usage: 3.3+ KB
  
```

Info()- inspects the DataFrame metadata. Useful for:

1. Listing feature names
2. Counting the number of rows
3. Identifying the data types of each feature
4. Discovering which column contains missing values"

Typical EDA

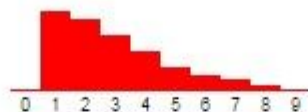
1. **Inspect data types:** Identify data types (e.g., numerical, categorical) and apply preprocessing techniques (e.g., scaling numerical features, encoding categorical variables).
2. **Calculate descriptive statistics:** Find measures of central tendency (e.g., mean, median) and variability (e.g., sd). Identify correlations to explore relationships between variables.
3. **Evaluate the distribution shape:** Analyze variable distributions to see if they follow expected patterns (e.g., normal distribution) are skewed (e.g., use log transformation).

Typical EDA

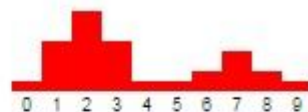
1. **Inspect data types:** Identify data types (e.g., numerical, categorical) and apply preprocessing techniques (e.g., scaling numerical features, encoding categorical variables).
2. **Calculate descriptive statistics:** Find measures of central tendency (e.g., mean, median) and variability (e.g., sd). Identify correlations to explore relationships between variables.
3. **Evaluate the distribution shape:** Analyze variable distributions to see if they follow expected patterns (e.g., normal distribution) are skewed (e.g., use log transformation).



Symmetric, unimodal,
bell-shaped



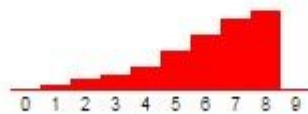
Skewed right



Non-symmetric, bimodal



Uniform



Skewed left



Symmetric, bimodal

Typical EDA

1. **Inspect data types:** Identify data types (e.g., numerical, categorical) and apply preprocessing techniques (e.g., scaling numerical features, encoding categorical variables).
2. **Calculate descriptive statistics:** Find measures of central tendency (e.g., mean, median) and variability (e.g., sd). Identify correlations to explore relationships between variables.
3. **Evaluate the distribution shape:** Analyze variable distributions to see if they follow expected patterns (e.g., normal distribution) are skewed (e.g., use log transformation).
4. **Utilize data visualization:** Graphs like histograms, box plots, and scatter plots reveal patterns, outliers, and relationships.

median (Q2/50th Percentile): the middle value of the dataset.

first quartile (Q1/25th Percentile): the middle number between the smallest number (not the “minimum”) and the median of the dataset.

third quartile (Q3/75th Percentile): the middle value between the median and the highest value (not the “maximum”) of the dataset.

interquartile range (IQR): 25th to the 75th percentile.

whiskers (shown in blue)

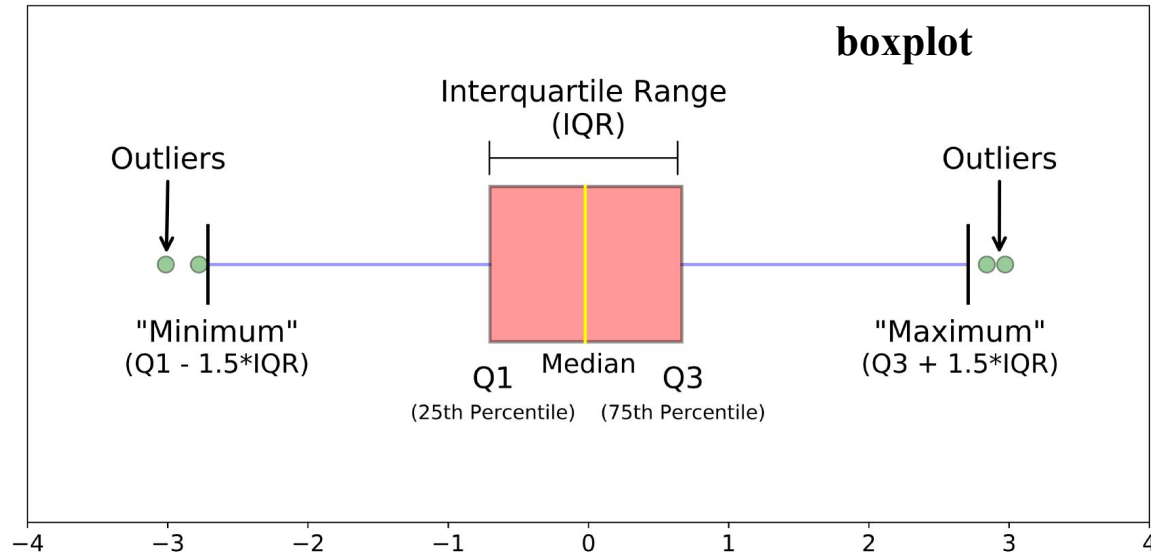
outliers (shown as green circles)

“maximum”: $Q3 + 1.5 * IQR$

“minimum”: $Q1 - 1.5 * IQR$

EDA - demonstration

Utilize data visualization

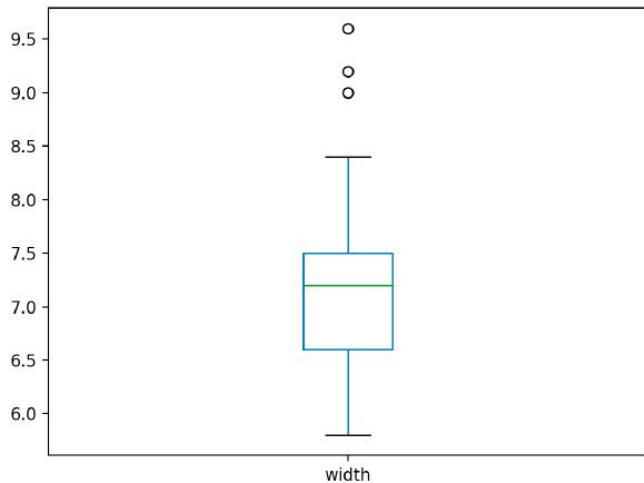


EDA - demonstration

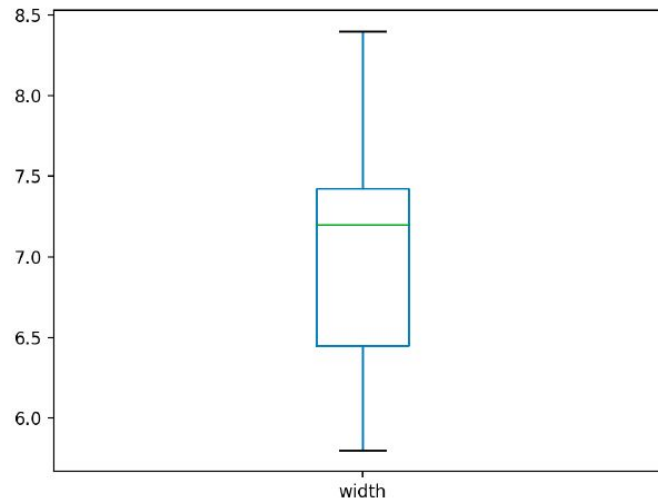
Utilize data visualization

```
fruits['width'].plot.box()
```

```
q95 = fruits['width'].quantile(.95)
fruits[fruits['width'] < q95]['width'].plot.box()
```

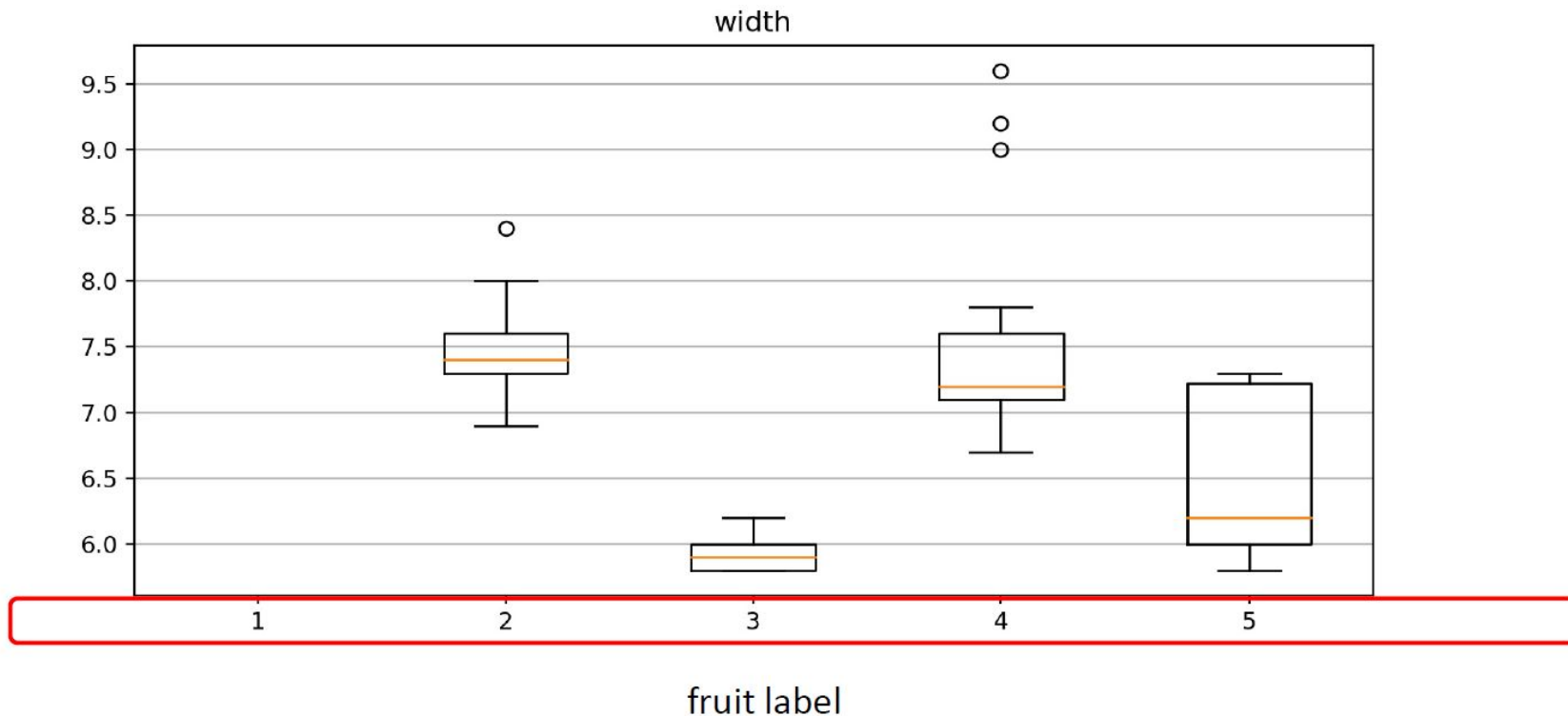


Remove



EDA - demonstration

Utilize data visualization (Bivariate Analysis)



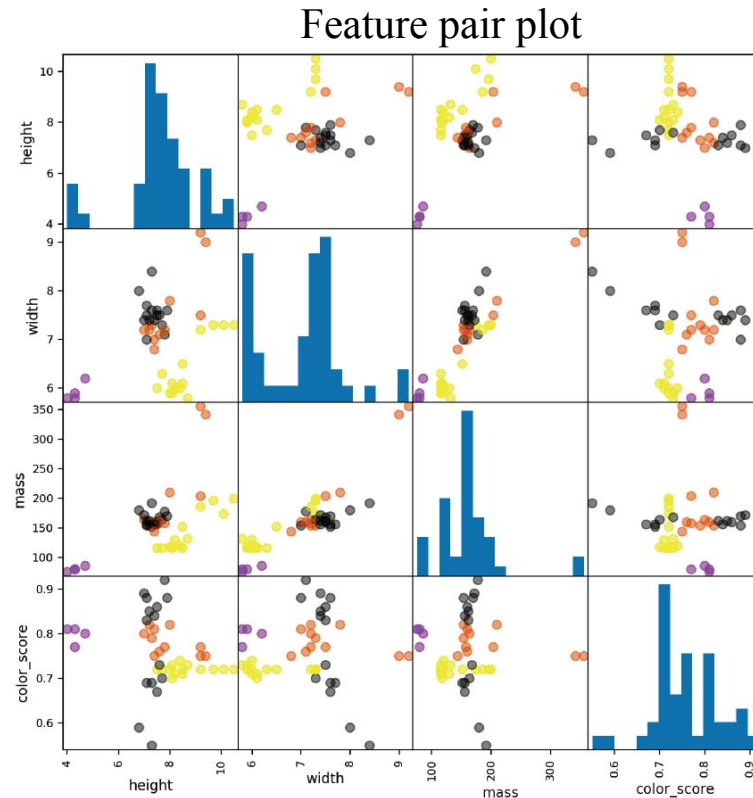
EDA - demonstration

Utilize data visualization (Bivariate Analysis)

Feature space - the representation of an object using specific features that are in certain columns of the data.

Using a pair plot we may:

- Get a glimpse of the distribution of each feature
- See which pairs of features help separate the dataset best
- Get a better idea how likely it is we could do at predicting the different classes



Typical EDA

1. **Inspect data types:** Identify data types (e.g., numerical, categorical) and apply preprocessing techniques (e.g., scaling numerical features, encoding categorical variables).
2. **Calculate descriptive statistics:** Find measures of central tendency (e.g., mean, median) and variability (e.g., sd). Identify correlations to explore relationships between variables.
3. **Evaluate the distribution shape:** Analyze variable distributions to see if they follow expected patterns (e.g., normal distribution) are skewed (e.g., use log transformation).
4. **Utilize data visualization:** Graphs like histograms, box plots, and scatter plots reveal patterns, outliers, and relationships.
5. **Detect inconsistencies and anomalies:** Identify missing data, outliers, and mixed data types to decide if they need correction or removal.

```
fruits.head(20)
```

	fruit_label	fruit_name	fruit_subtype	mass	width	height	color_score
0	1	apple	granny_smith	192	8.4	7.3	0.55
1	1	apple	granny_smith	180	8.0	6.8	0.59
2	1	apple	granny_smith	176	7.4	7.2	173.00
3	2	mandarin	mandarin	86	6.2	4.7	0.80
4	2	mandarin	mandarin	84	6.0	4.6	0.79
5	2	mandarin	apple	80	5.8	4.3	0.77
6	2	mandarin	mandarin	80	5.9	4.3	0.81
7	2	mandarin	mandarin	76	5.8	4.0	0.81
8	1	apple	braeburn	178	7.1	7.8	0.92
9	1	apple	braeburn	172	7.4	7.0	0.89
10	1	apple	braeburn		6.9	7.3	0.93
11	1	apple	braeburn		7.1	7.6	0.92
12	1	apple	braeburn		7.0	7.1	0.88
13	1	apple	golden_delicious		7.3	7.7	0.70
14	1	apple	golden_delicious		7.6	7.3	0.69
15	1	apple	golden_delicious		7.7	7.1	0.69
16	1	apple	golden_delicious	156	7.6	7.5	0.67
17	1	apple	golden_delicious	168	7.5	7.6	0.73
18	1	apple	cripps_pink	162	7.5	7.1	0.83
19	2	apple	cripps_pink	162	7.4	7.2	0.85

EDA - demonstration

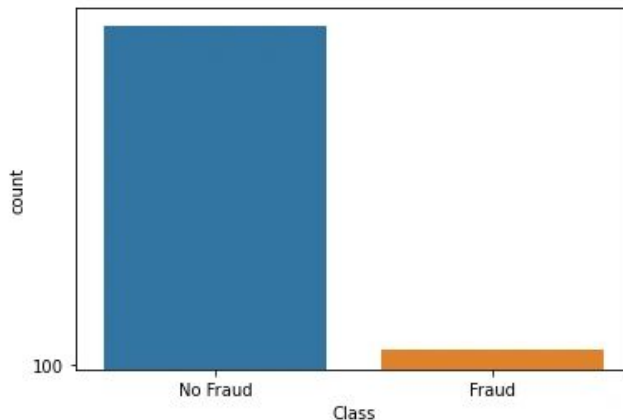
5. Detect inconsistencies and anomalies

Typical EDA

1. **Inspect data types:** Identify data types (e.g., numerical, categorical) and apply preprocessing techniques (e.g., scaling numerical features, encoding categorical variables).
2. **Calculate descriptive statistics:** Find measures of central tendency (e.g., mean, median) and variability (e.g., sd). Identify correlations to explore relationships between variables.
3. **Evaluate the distribution shape:** Analyze variable distributions to see if they follow expected patterns (e.g., normal distribution) are skewed (e.g., use log transformation).
4. **Utilize data visualization:** Graphs like histograms, box plots, and scatter plots reveal patterns, outliers, and relationships.
5. **Detect inconsistencies and anomalies:** Identify missing data, outliers, and mixed data types to decide if they need correction or removal.
6. **Assess the representation of each class:** Check if there are enough examples for each class (e.g., handle imbalanced datasets with oversampling).

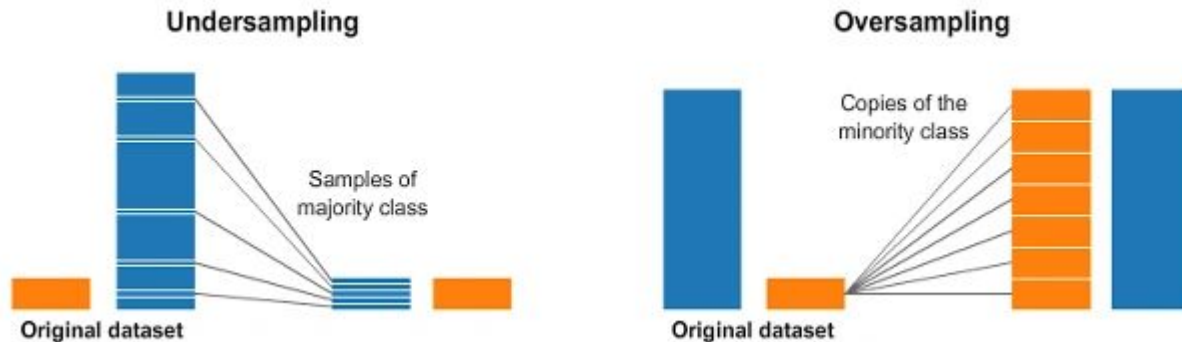
Typical EDA

One of the widely adopted imbalance classification techniques for dealing with highly unbalanced datasets is called **resampling**. It consists of removing samples from the majority class (under-sampling) and/or adding more examples from the minority class (over-sampling).



Typical EDA

One of the widely adopted imbalance classification techniques for dealing with highly unbalanced datasets is called **resampling**. It consists of removing samples from the majority class (under-sampling) and/or adding more examples from the minority class (over-sampling).



Typical EDA

1. **Inspect data types:** Identify data types (e.g., numerical, categorical) and apply preprocessing techniques (e.g., scaling numerical features, encoding categorical variables).
2. **Calculate descriptive statistics:** Find measures of central tendency (e.g., mean, median) and variability (e.g., sd). Identify correlations to explore relationships between variables.
3. **Evaluate the distribution shape:** Analyze variable distributions to see if they follow expected patterns (e.g., normal distribution) are skewed (e.g., use log transformation).
4. **Utilize data visualization:** Graphs like histograms, box plots, and scatter plots reveal patterns, outliers, and relationships.
5. **Detect inconsistencies and anomalies:** Identify missing data, outliers, and mixed data types to decide if they need correction or removal.
6. **Assess the representation of each class:** Check if there are enough examples for each class (e.g., handle imbalanced datasets with oversampling).
7. **Feature engineering:** Create or transform features to improve data representation (e.g., interaction terms or aggregating temporal data).

Typical EDA

Why Feature engineering is important?

- **Improves Predictive Power:** Well-crafted features can make patterns in the data more visible to machine learning models.
- **Reduces Complexity:** Transforming raw data into meaningful features simplifies the learning process for algorithms.
- **Addresses Domain-Specific Insights:** Feature engineering incorporates knowledge about the problem domain, enabling the model to leverage contextual information.

Feature Engineering Examples:

- Dates: extract year, month, dayofweek, is_weekend
- Interactions: e.g., income_per_age, weight / height²
- Binning: e.g., age_group = child / young / adult / senior

Additional reading

Recommended reading for next time: The Foundations of Algorithmic Bias -

<https://www.approximatelycorrect.com/2016/11/07/the-foundations-of-algorithmic-bias/>

- Processing Data To Improve Machine Learning Models Accuracy -

<https://medium.com/fintechexplained/processing-data-to-improve-machine-learning-models-accuracy-de17c655dc8e>

- Exploration of the “adults” notebook -

<https://medium.datadriveninvestor.com/introduction-to-exploratory-data-analysis-682eb64063ff>

- EDA and Outliers - <https://www.kaggle.com/code/life2short/eda-and-outliers>

- Official Pandas documentation - <https://pandas.pydata.org/pandas-docs/stable/index.html>

Pandas cheatsheet PDF

https://github.com/pandas-dev/pandas/blob/main/doc/cheatsheet/Pandas_Cheat_Sheet.pdf

Hands on !



Open Google colab notebook -
ML L1-EDA Student.ipynb

Google Colab

Create or Upload Notebook



Open Google Colab

<https://colab.research.google.com/>



Upload Datasets

Import the datasets you'll be working with into the notebook.



Save

Save as an .ipynb file so that next time, you can upload it by following step 1.

